1 what is signal? state the features of signal

-> is a notification sent to a process to notify it of an event or exception

features

asynchronous notification

inter process communication

predefined types

limited info transfer

3 list the properties of Linux operating system

-> Linux is an open source os

multi-user

multitasking

security

portability

d) Explain Kill() function with syntax.

-> used to send a specific signal to a process or a group of process

syntax : #include<sys/types.h>

#include<signal.h>

int kill(pid_t pid,int sig);

pid=process group id

sig=integer value of the signal to send

the function returns 0 on success

-1 on error

a) Explain the syntax of open ( ) system call.

-> used to open or create a file in the system.read(),write(),close().
   syntax : #include<sys/types.h>
            #include<sys/stat.h>
            int open (const char *pathname,int flags)
            pathname= the path of the file to be opend or created
e) What is Buffer cache?
-> a buffer cache / a disk buffer cache isa prtion of main memory (RAM) used
   by the os to store copies of data blocks
   goal-speed to input output request
   when a program request data,the os checks the cache
f) Define nice() system call.
-> it is used to change the priority of a process
   high value from -20 to 19
   19 lowest value
   syntax - #include<unistd.h>
            it nice(int inc);
            inc=priority value
f) What is Process ID?
-> is a unique integer number assigned by the os kerent to each process
   the os uses pid to identify and manage process
h) What is Sticky bit?

-> is an unix permission flag
   when set on a directory,restrics file deletion within the directory

- Explain swapping in detail

-> swapping is a memory management technique in an os
   temporaily moves an entire process from the main memory to
   a secondary storage to free up RAM for other processes
   the area on the secondary storage used for the swap space
   two main steps
   swap out :moving a process from main memory to the swap space
            ram is full
   swap in :moving a process back from swap space to main memory
            so it can continue execution
   swapping increases the degree of multiprogramming

- explain the structure of regular file with diagram

-> inode-the inode is a data structure that stores all the metadata
        about the file except its name and

actual data
key info:
file type(regular directory)
permissions and ownership(user ID,group ID)

timestamps(creations,modification,last access)
size of the file in bytes
poniters(or block address)
data blocks- actual disk blocks
store the contents of the file
the inode points to this blocks

19 explain different types of memory regions found in every process

-> text segment - contains the programs executable code
data segment - contains initialized global and static variables
BSS segment - contains unitialized global and static variables
Heap - a dynamic memory allocation area
stack - a region for storing function call frames and local variables

- What are common sections of process?

-> text segments:this section contains the compiled machine code of the program
it is read only mode

data segment :stores global and static variable
initialized data segment :
contains global and static variables that are initialized
eg.int x=5
uninitialized data segment :
contains global and local variable that are decleared
but not initialized
eg.static int y
stack segment:storing local variables,manageing function calls
it follows the last in first out
heap segment :used for dynamic memory allocation during runtime
using functions like malloc,calloc,realloc

- What is environment variable? List any two.

-> an environment variable is a dynamic named value
they exist within the environment of a process
used to store info
list :
PATH:contains a colon separated list of

directories
HOME:stores the path to the current users home directory

- List the characteristics of Unix O.S

-> multiuser:it allows multiple users to access
multitasking:it can run multiple processes or programs concurrently
portability:easy to port to different hardware platforms
hierarchical file system:it uses a tree like hierarachical structure
pipes and redirection:it provides powerful mechanisms
security:user authentication

- Write a short note on buffer headers & buffer pool.

-> buffer pool:
a buffer pool is a region of main memory that
the kernel uses to temporaily store data
this acts cache improves I/O performance
when data is read from or written to disk

it first goes into a buffer in the pool

buffer header:
each buffer in the pool has an associated buffer header
data structure containing all the metadata
to manage that specific buffer
the header links the buffer

- Windows is portable operating system, Discuss.

-> windows was not designed with the portability
morden windows is a high degree of portability

- Explain four major data structures to support demand paging.

-> page table
frame table
secondary memory data structure
page fault handling replacement data structure

- Explain kill() & raise() functions.

raise()-used to send a signal to current process
raise a signal with in the program

- Explain the system calls SIGSTOP and SIGTERM.

-> SIGSTOP:

this signal used to stop process execution

it is a powerful signal

SIGTERM:

used to request a process to terminate

- context of process

cpu registers

process ID(PID) and parent process ppid

memory management info

I/o status info

signal handling info

- What is use of syncs,fsync,and fdata sync() functions.

-> sync()-all data buffered in memory to be written to the disk

it initiates the writes

fsync(int fd)-all data and metadata associated with the fd

fdatasync(int fd)-synchronize the files data to the disk

it faster than fsync()

- State difference between fchown() and lchown() function.

-> fchown()-fchown(int fd,uid_t owner,gid_t group)

it operates on the file that the

fd
it opend via symbolic link
used to change the owner and group of file discriptor
lchown()-lchown(const char *path,uid_t owner,gid_t group)
used to change the owner and group specified by path
if specified path is a symbolic link,lchown() changes
the ownership of the symbolic link itself

- Explain alarm () & pause () Functions.

-> alarm(),pause() functions are POSIX system calls
used for handling time-based
process pausing in UNIX systems
alarm(unsigned int sec):
if seconds is 0,any pending alarm is cancelled
by default SIGALRM terminates the process
this function returns the number of seconds
pause(void):suspend process execution

- Explain fork() & vfork() system call with example.

-> fork()-creats a new process
both parent and child process execute concurrently

```
   eg.    int main()
         {
            pid_t pid=fork();
            if(pid==0)
                printf("child process
PID=%d,getpid());
            else
                printf("parent process PID
=%d,child PID))
         }
   vfork()-creates a new child process
          child process share the parent
space
   eg.    int main()
         {
            pid_t pid=vfork();
            if(pid==0)
                printf("child process
PID=%d,getpid());
            else
                printf("parent process PID
=%d,child PID))
         }




  }
```

8 explain first scenario of buffer pool

-> is a cache of frequently accessed data in memory
9 under which circumstances the process is swapped out
-> a process is swapped out when it is moved from main memory to
   secondary storage to free up memory for other processes
20 what is internal and external fragmentation
-> internal - wasted space within a block of memory due to alignment requirements
   external - wasted space between blocks ofmemory due to non contiguous allocation
14 write fields of Inode
-> file type and permissions
   file owners user ID and group ID
   file size
   creations,modification,access
   direct,indirect,doubly indirect
23 explain different types of files with respect to Linux operating system
-> regular files - files containing data or executable code
   directories - files that contain other files or directories
   devcice files- special files that represent hardware devices
   symbolic links - files that point to

other files or directories
11 justify the following :No process can pre-empt another executing in kernel
-> data integrity and race condtion
   simplified kernel design
   predictable execution
15 explain three levels of UNIX operating system
-> kernel - the core of the os,responsible for managing hardware resources
            acts as an interface between the hardware the user program
            it manage memory location,processes,file systems,i/o device
   system calls - interfaces between user level programs and the kernel
   user level programs - applications and utilities that run on top of the kernel
   hardware layer : it is at the bottom,and the kerenel interacts with it directly
                    containing CPU,memory(RAM),hard disks
18 explain following system calls
-> memory functions
   I)Memset() - sets a block of memory to a specified value
                void memset(void ptr,int value,size_t num)
   II)Memchr() - searches for the first

occurrences of a character in a block of memory

void memchr(void ptr,int value,size_t num)

III)Memcmp() - compares two blocks of memory

void memcmp(void ptr,int value,size_t num)

IV)Memmove() - moves a block of memory from one location to another

void memmove(void ptr,int value,size_t num)

22 explain read(),write(),readv(),writev() with syntax and example

-> file operations

read() - reads data from a file descriptor

syntax: read(int fd,void buffer)

eg int main()
char b[10]
int fd=open("test.txt");
if(fd!=-1){
b_read=read(fd,b)
print("read=%s",b_read)}

write() - writes data to a file descriptor

syntax: write(int fd,void buffer)

eg int main()

```
                char b[10]="hello"
                int fd=open("test.txt");
                if(fd!=-1){
                b_read=write(fd,b)
                print("read=%s",b_read)}
   readv() - reads data from a file
descriptor into multiple buffers
            syntax: readv(int fd,void
buffer)
            eg
   writev() - writes data from multiple
buffers to a file descriptor
            syntax: writev(int fd,void
buffer)
            eg
12 explain malloc(),calloc(),realloc() and
free() functions
   or List the different memory allocation
functions.
-> memory management functions
   malloc() - memory allocation
              allocates a block of memory of
a specified size
              syntax:void
malloc(size_t,size)
   calloc() - continuous allocation
              allocates an array of memory
blocks all initialized to zero
              syntax:void calloc(size_t
```

n,size_t size)
   realloc() -reallocation
               resizes a block of memory
previously allocated using malloc() or
calloc()
               void realloc(void ptr,sze_t
size)
   free() -   releases a block of memory
back to the system
               void free(void *ptr)